Student Feedback Intelligence

Mid-Term Evaluation Guide

Everything built so far (Weeks 1–6), in plain words, and how to show it

---

SNLP Course Project 2026 • Group 5

E029 · E030 · E031 · E033 · E034 · E035

Prepared 18 October 2026 for the mid-term evaluation on 19 October
Sections 1–6 explain the project. Section 7 is the step-by-step demo. Section 9 has likely questions.

1. The Project in One Minute

The problem. Universities collect written feedback from students, but teachers usually see only an average rating, such as 3.8 out of 5. An average does not say *what* to fix: was the teaching unclear, the pace too fast, or the workload too heavy? The answer is in the written comments, and nobody has time to read hundreds of them.

What our tool does. It reads each comment and answers two questions for every part of the course the comment talks about:

1. **Which part (aspect)?** One of 8: teaching, faculty interaction, course content, pace, assessment, workload, resources, other.
2. **How does the student feel about it (polarity)?** Positive, negative, neutral (mentioned without an opinion) or conflict (both good and bad).

The handout's own example, *"The lectures were very informative, but the pace was too fast and the assignments were difficult."*, as our tool answers it:

Aspect	Feeling	Words the tool points to
teaching	positive	"The lectures were very informative"
pace	negative	"the pace was too fast"
assessment	negative	"the assignments were difficult"

It then summarises a whole course: top strengths, top concerns, "emerging" concerns that only a few students raised, and real quotes as evidence.

Why it is an "NLP asset" and not just an app. It runs as a service with a documented web API. Any other group's application can send it comments and get structured answers back, without knowing how it works inside. The handout's example of this is Group 6 (Early Warning), which could receive course-level workload concerns.

One sentence for the panel: *"We built a reusable service that turns free-text student feedback into aspect-level sentiment, with the evidence words and an honest confidence score, and we measured it against a simple baseline on comments it had never seen."*

2. Words You Need

Term	What it means here
Aspect	The part of the course a comment talks about (8 of them: teaching, faculty interaction, course content, pace, assessment, workload, resources, other)
Polarity / sentiment	How the student feels about that aspect: positive, negative, neutral, or conflict (both)
ABSA	Aspect-based sentiment analysis: finding the aspects *and* the feeling for each. Plain sentiment analysis gives one label per comment, which is wrong for "good lectures, bad pace"
Baseline	The simple first model we compare against: TF-IDF + logistic regression. It counts which words appear and learns which words go with which label
Transformer / DeBERTa	A large pretrained language model (184 million parameters) that reads whole sentences with context. We used DeBERTa-v3-base
Fine-tuning	Further training a pretrained model on our own labelled examples so it learns our task
Attention	The model's way of focusing on certain words. We gave each of the 8 aspects its own "spotlight", so the pace spotlight looks at "too fast" and the teaching spotlight looks at "informative"
Evidence span	The words the tool points to as the reason for each answer
Train / validation / test	The 5,998 comments split 70/15/15. Train = learn. Validation = tune our choices. Test = the final exam, opened only to measure, never to choose anything
Tuple-F1	Our main score, from 0 to 1. An answer counts only if both the aspect *and* the feeling are right. F1 balances how many of our answers are right (precision) with how many of the true answers we found (recall)
Confidence interval	The range the score would likely move within on a different sample of comments, e.g. 0.704 [0.683, 0.723]
Calibration	Making the confidence numbers honest, so that "0.9 sure" really means right about 9 times in 10
API	A doorway other programs use: send text as JSON over the web, get JSON back

3. The Data

Source: EduRABSA, a public research dataset (Hua et al., 2025) of 6,500 real student reviews:

- course reviews from the University of Waterloo;
- teacher reviews from RateMyProfessor;
- university reviews from Exeter.

An expert already labelled every review with its aspects and feelings. We kept the **5,998 course and teacher reviews** and dropped the 500 university-wide reviews (they are about hostels, the city and fees, which our tool does not cover).

Why not write our own comments? If the six of us wrote the comments *and* labelled them, the model would learn our writing style, and its score would mean nothing. Real public data also avoids any privacy problem with our classmates' feedback.

Converting the labels. EduRABSA uses 24 categories; we use 8. We wrote fixed rules for the conversion. For example, "Staff – Attitude" becomes faculty interaction, and "Course – Overall" ("Good course") gets no aspect at all. Two rules matter:

- **Pace:** EduRABSA has no pace category, so a label becomes pace when its words are about speed, such as "too fast" or "rushed".
- **Lectures:** EduRABSA files lectures under "learning activity". We moved them to teaching in week 4, which made the handout's own example sentences come out right.

Split: 4,198 comments to train, 900 to validate, 900 to test. The split was fixed once and never changed. The licence (CC BY-NC-SA 4.0) allows educational use with credit.

4. What We Built, Week by Week

Week	What we did	Result
1–2	Chose the dataset and converted its labels; wrote labelling guidelines; built the baseline (TF-IDF + logistic regression) and the evaluation code	Baseline score **0.454** on the test set (blind guessing: 0.216)
3	Built the main model: DeBERTa with one attention "spotlight" per aspect, also trained to point at the evidence words. Wrapped it as pipeline v1: comment in, structured JSON out	**0.712** on the validation set (baseline on the same comments: 0.476)
4	The fair final comparison on the test set. Added a middle model (sentence embeddings, the handout's own example), tuned the decision settings on validation, made evidence readable phrases, fixed the lectures labelling	**0.704** vs **0.452** for the baseline; middle model 0.494; evidence matches the human-marked words 55% of the time (was 12%)
5	Honesty checks: calibrated the confidence numbers, broke results down by aspect and by type of comment, tested typos and messy text, wrote 32 probe sentences for specific behaviours, read 40 mistakes by hand	Confidence error for whole-comment mood fell from 0.116 to 0.021; typos cost only 0.026; 17 of 32 probes fully right
6	Turned it into a real web API: 5 endpoints, an automatic documentation page, error codes, a course report with emerging concerns, example code for other groups	31 automated tests pass; about 45–61 ms per request

5. The Results in Plain Words

Headline: on 900 comments it had never seen, our main score is **0.704**, against **0.452** for the simple baseline. The difference (+0.25) is far larger than the uncertainty in either number.

Why ours is better. The baseline reads the whole comment at once, so in "good lectures but terrible pace" it gave both aspects the same overall mood. Our model's per-aspect spotlights read each part separately.

Is it just because we used a big model? No. A pretrained model used *without* training it on our task (the week 4 middle model) scored only 0.494, barely above the baseline. The gain comes from fine-tuning plus the per-aspect attention.

Where it is strong:

- teaching (0.80) and faculty interaction (0.80);
- telling positive from negative: they were swapped only 16 times in 1,187 judgements;
- short, explicit comments such as "Too fast." (6 of 6 probes right).

Where it is weak:

- resources (0.49) and other (0.54);

- neutral and mixed opinions, which get pulled to one side;
- very long reviews (0.61 above 80 words) and sentences with "but" (0.65).

Honest confidence. After calibration, when the tool says 0.9 it is right about 90% of the time. On the handout sentence it is 98% sure about pace and assignments, but only 50% sure the lectures remark is positive, and it says so.

We did not cheat the test. Every choice was made on the validation set and then frozen. The test set was opened only to measure, and every opening is logged in a file.

6. What It Gets Wrong, and Why

Weakness	Example	Why
Sarcasm	"Oh great, another surprise quiz on Monday." → nothing found (0 of 4 probes)	It reads words literally; the training reviews rarely use sarcasm
Hinglish	"Sir ka padhane ka tareeka bahut accha hai." → nothing found (0 of 5)	Trained only on English reviews
Mixed and neutral opinions	"nice … but disorganized" → only positive	These are rare in training (8–9% of labels)
Blames teaching	"The exams were not fair." → assessment negative (right) plus teaching negative (wrong)	Half the training data are reviews *of teachers*, so it learned that negativity usually concerns the teacher. A fairness issue: it could make reports harsher on instructors
Long reviews	80+ words score 0.61	Many aspects compete; minor ones get missed
Privacy	Names with no title ("Kathy was great") are not removed	The redaction rule only catches titled names, emails, phone and roll numbers

The demo must show at least one failure (handout, Part G). Use the sarcasm or the Hinglish sentence (Section 7, step 5): it is easy to understand and easy to explain.

7. How to Show It Tomorrow

Before you start:

- Use **Krish's laptop**. The trained model (738 MB) is only on its D: drive, so the live demo cannot run on another laptop.
- Keep the laptop on charger and close heavy apps. The demo needs no internet, except that the `/docs` page loads its styling online. If Wi-Fi fails, the terminal demo (step 6) still works.

Steps, on the PC in PowerShell:

1. Go to the project folder: `cd "C:\Users\Krish\SynologyDrive\NMIMS\SEM 11\Projects\SNLP Sem 11 Project\Group5_Student_Feedback_Intelligence (till week 5-6)\Group5_Student_Feedback_Intelligence"`
2. Start the service, and leave this window open: `D:\snlp-feedback\venv\Scripts\python.exe src\api.py` Wait about 15 seconds for "Uvicorn running on http://127.0.0.1:8000".
3. In the browser, open `http://127.0.0.1:8000/docs`. Show the 5 endpoints: this page *is* the reusable asset, and it is how another group would discover and try it.
4. Click **POST /v1/analyze-feedback → Try it out**. The handout sentence is already filled in. Click **Execute**, then explain the three aspects, the feelings, the evidence words and the confidence scores.
5. **Show a failure.** Replace the text with `Oh great, another surprise quiz on Monday.` and click **Execute**. Point out that `aspects` comes back empty, and explain that sarcasm is read literally.
6. Open a second PowerShell window, run the same `cd` as step 1, then: `python examples\demo_course_report.py` This sends 150 real reviews and prints a course report. Explain the top strengths (teaching, course content, faculty interaction), the top concern (workload), the emerging concern (pace: only 3 mentions, all negative), and the caveats.
7. (Optional) In the browser, run **GET /v1/model-card**: it lists the training data, the test results and the known failures, so another group knows when *not* to trust the tool.
8. (Optional) In the second window, run the automated tests: `D:\snlp-feedback\venv\Scripts\python.exe -m pytest tests -q` → "31 passed".
9. When finished, click the first window and press **Ctrl+C** to stop the service.

If the service will not start: in the project folder, run `D:\snlp-feedback\venv\Scripts\python.exe src\pipeline.py "The lectures were good but the pace was fast."` It prints the same answer without the web part. If that also fails, present from the week reports (results tables).

8. Suggested Order (about 15 minutes)

1. **Problem and example** (1 min): Section 1, with the handout sentence.
2. **Data** (2 min): why a public expert-labelled dataset, the 8 aspects, the split.
3. **Method** (3 min): baseline → middle model → fine-tuned DeBERTa with per-aspect attention, and why it wins.
4. **Results and honesty** (2 min): 0.704 vs 0.452, the frozen test set, calibration.
5. **Live demo** (4 min): Section 7, steps 3–6, including the failure.
6. **Reuse and what is next** (2 min): the API, the Group 6 connection, weeks 7–8.

9. Questions You May Be Asked

Why not just use ChatGPT? The handout rules out a wrapper around a chat model. We also need a score we can measure and compare, results that do not depend on a paid external service, no student text sent outside the institution, and evidence for every answer. Our model runs locally on a laptop.

Why this dataset? It is real student language, public, education-specific and already labelled by an expert. Self-written comments would make the score meaningless, and real classmates' feedback would raise privacy issues.

Why only 8 aspects? Enough to separate the things a coordinator can act on (teaching vs pace vs workload), but few enough that each has hundreds of examples to learn from.

Why tuple-F1 and not accuracy? A comment has several answers, not one. Tuple-F1 only gives credit when both the aspect and its feeling are right, and it punishes both missed and invented aspects.

How do you know you did not tune on the test set? Every setting was chosen on the validation set and frozen with a fingerprint; the test script refuses to run if the settings changed afterwards, and every opening of the test set is logged. All models drop by a similar small amount from validation to test, including the baseline, which nobody tuned.

What is the baseline and why does it lose? Word counts (TF-IDF) with logistic regression. It cannot tell which words belong to which aspect, so mixed reviews get one mood for everything.

Why DeBERTa? It is a strong pretrained model that understands context and fits on a 4 GB laptop GPU. Two tricks made it fit: freezing its word-embedding table, and capping how much text goes in each training batch.

What does calibration change? Only the confidence numbers, never the answers. It makes "0.9" mean about 90%.

How would another group use it? Start the service and send JSON to `POST /v1/analyze-feedback` or the batch endpoint. `docs/API.md` explains everything with examples; `examples/client_example.py` is ready-made code. Group 6 could take workload or pace concerns as course-level advisory signals.

What is the biggest weakness? Neutral and mixed opinions, sarcasm, Hinglish, and a tendency to add "teaching: negative" to negative comments.

What about privacy and fairness?

- Emails, phone numbers, roll numbers and titled names are removed before analysis and from report quotes.
- Output is course-level and advisory only, and it must never be used to judge one student or rank teachers.
- Reports with fewer than 20 comments carry a warning.

How fast is it? About 45–61 ms per comment through the API on the laptop GPU, and about 67 ms on a CPU. Training takes about 6 minutes per run.

What is left?

- **Week 7:** connect to another group (Group 6), a responsible-AI statement, a demo interface.
- **Week 8:** final report, slides, code freeze.

10. Where Everything Is

Project folder: `NMIMS\SEM 11\Projects\SNLP Sem 11 Project\`

Item	Location
Latest code (use this)	`Group5_Student_Feedback_Intelligence (till week 5-6)\Group5_Student_Feedback_Intelligence\`
Earlier snapshots	`(till week 1-2)`, `(till week 3)`, `(till week 4)` folders
Weekly reports	`docs\Group5_Week1-2_Report_v3`, `Group5_Week3_Report`, `Group5_Week4_Report`, `Group5_Week5-6_Report` (each .docx, .pdf, .md)

Item	Location
How to use the API	`docs\API.md`
The service	`src\api.py`
The pipeline (all the NLP)	`src\pipeline.py`, with the model in `src\transformer_absa.py`
Demo scripts	`examples\demo_course_report.py`, `examples\client_example.py`
All measured results	`results\` (e.g. `week5_eval.json`, `week4_d1.1_test_eval.json`)
Trained model (not in the folder)	`D:\snlp-feedback\checkpoints\deberta-v3-base_sw0.5_s42_d1.1` on Krish's laptop